

1. Einleitung und Übersicht

Der Zugriff auf und die Verwaltung von *Information* spielt eine immer wichtiger werdende Rolle in der heutigen Gesellschaft – sei es für Unternehmen, Politiker, Wissenschaftler, Verwaltungen, etc. Es wird geschätzt, dass sich die „Informationsmenge“ derzeit alle 5 Jahre verdoppelt – zumindest trifft dies für die in Büchern abgelegte Information nach Statistiken der amerikanischen Library of Congress zu. Während in früheren Zeiten der Großteil der Information auf Papier abgelegt war, werden wir heute von einer elektronischen Informationsflut „überrollt“. Deshalb gewinnen *Datenbankverwaltungssysteme* (engl. *database management systems*, abgek. *DBMS*) eine immer größere Bedeutung. Heute findet sich kaum noch eine größere Organisation oder ein größeres Unternehmen, das nicht ein DBMS für die Informationsverwaltung einsetzt. Man denke etwa an Banken, Versicherungen, Flugunternehmen und Universitätsverwaltungen (um unsere Beispielanwendung dieses Buchs schon mal zu erwähnen).

Ein Datenbankverwaltungssystem besteht aus einer Menge von *Daten* und den zur Datenverarbeitung notwendigen Programmen:

- Die gespeicherten Daten werden oft als *Datenbasis* bezeichnet. Die Datenbasis enthält die miteinander in Beziehung stehenden Informationseinheiten, die zur Kontrolle und Steuerung eines Aufgabenbereichs (evtl. eines ganzen Unternehmens) notwendig sind.
- Die Gesamtheit der Programme zum Zugriff auf die Datenbasis, zur Kontrolle der Konsistenz und zur Modifikation der Daten wird als *Datenbankverwaltungssystem* bezeichnet.

Oft werden diese Komponenten aber auch weniger scharf getrennt, so dass man mit Datenbankverwaltungssystemen (oder kürzer Datenbanksystemen) sowohl die Datenbasis als auch die Verwaltungsprozesse der Datenbasis meint.

1.1 Motivation für den Einsatz eines DBMS

Es gibt heutzutage in den meisten Unternehmen und Organisationen keine Alternative mehr zum Einsatz eines DBMS für die Informationsverarbeitung. Wir wollen uns dies anhand der Probleme verdeutlichen, die ohne Nutzung eines einheitlichen, die gesamte Informationsverarbeitung abdeckenden DBMS, auftreten würden. Die im allgemeinen auf vielfältige Art miteinander in Beziehung stehenden Daten müssten dann entweder auf Papier (Karteikästen, Aktenordner, etc.) oder in isolierten Computer-Dateien abgelegt werden. Dies führt zu folgenden schwerwiegenden Problemen:

Redundanz und Inkonsistenz Wenn Daten in isolierten Dateien (oder andersartigen isolierten Archiven) gehalten werden, müssen dieselben Informationen bezüglich eines Anwendungsobjekts oft mehrfach, d.h. redundant, gespeichert werden.

Man denke etwa an die Adressinformation der Studenten einer Universität. Diese Information wird sicherlich in der Studentenverwaltung, aber auch in den jeweiligen Fakultäten benötigt. Bei Änderungen kann es dann zu Inkonsistenzen führen, wenn nur eine Kopie der Daten geändert wird, die andere aber noch im veralteten Zustand beibehalten wird. In einem globalen, integrierten DBMS wird diese Art von unkontrollierter Redundanz vermieden.

Beschränkte Zugriffsmöglichkeiten Es ist schwer, wenn nicht sogar unmöglich, die in isolierten Dateien abgelegten Daten miteinander zu „verknüpfen“, d.h. Information aus einer Datei mit anderen logisch verwandten Daten aus einer anderen Datei zu verknüpfen. Bei einem homogenen integrierten DBMS wird die gesamte Information einer Organisation einheitlich modelliert (wir sagen in demselben *Datenmodell*), so dass sich diese Daten sehr flexibel miteinander verknüpfen lassen.

Probleme des Mehrbenutzerbetriebs Die heutigen Dateisysteme bieten entweder gar keine oder nur sehr rudimentäre Kontrollmechanismen für den Mehrbenutzerbetrieb. Daten werden aber i.A. von sehr vielen Anwendern innerhalb und außerhalb der jeweiligen Organisation genutzt – als Beispiel sei ein Flugreservierungssystem genannt. Bei unkontrolliertem Zugriff kann es sehr leicht zu (äußerst) unerwünschten Anomalien kommen. Man denke etwa an das gleichzeitige unkontrollierte Editieren derselben Datei durch zwei Benutzer. Dabei kann es leicht vorkommen, dass die Änderungen des einen Benutzers von dem Benutzer, der die Datei zuletzt zurückschreibt, überschrieben werden. Dieses Phänomen nennt man im Englischen „lost update“.

Datenbankverwaltungssysteme bieten eine Mehrbenutzerkontrolle, die solche und noch andere unerwünschte Anomalien des Mehrbenutzerbetriebs ausschließen.

Verlust von Daten Wenn Daten in isolierten Dateien gehalten werden, wird die Wiederherstellung eines konsistenten – d.h. eines gemäß der realen Welt gültigen – Zustands der Gesamtinformationsmenge im Fehlerfall sehr schwierig. Im Allgemeinen bieten Dateisysteme bestenfalls die Möglichkeit einer periodisch durchgeführten Sicherung der Dateien. Datenverluste, die während der Bearbeitung von Dateien oder nach der letzten Sicherungskopie auftreten, sind i.A. nicht auszuschließen.

Datenbankverwaltungssysteme besitzen eine ausgefeilte Recoverykomponente, die den Benutzer für alle vorhersehbaren Fehlerfälle vor Datenverlust schützen soll.

Integritätsverletzung Je nach Anwendungsgebiet gibt es vielfältigste, sich global über mehrere Informationseinheiten erstreckende Integritätsbedingungen. Man denke im Universitätsbereich etwa an die Bedingung, dass Studenten erst die Pflichtseminare abgeschlossen haben müssen, bevor sie zur Prüfung zugelassen werden dürfen. Oder dass dieselbe Prüfung maximal zweimal wiederholt werden darf. Die Einhaltung derartiger Integritätsbedingungen ist bei der isolierten Speicherung der Informationseinheiten in verschiedenen Dateien sehr schwierig, da man zur Kontrolle Daten aus unterschiedlichen Dateien verknüpfen muß. Außerdem will man im Allgemeinen nicht nur die Konsistenzbedingungen überprüfen, sondern die Einhaltung erzwingen, d.h. bestimmte Datenverarbeitungsvorgänge sollen vom System „abgelehnt“ werden, falls sie zu einer Verletzung der Integrität führen werden. In

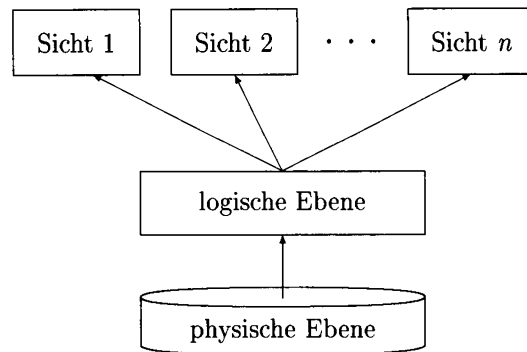


Abb. 1.1: Drei Abstraktionsebenen eines Datenbanksystems

Datenbankverwaltungssystemen werden Transaktionen (das sind die aus Benutzersicht atomaren Verarbeitungsvorgänge) nur dann vollzogen, wenn sie die Datenbasis in einen konsistenten Zustand überführen.

Sicherheitsprobleme Nicht alle Benutzer sollten Zugriff auf die gesamten gespeicherten Daten haben. Und erst recht sollten nur bestimmte ausgewählte Benutzer das *Privileg* haben, Daten zu ändern. Man denke etwa an die Information, die zu den Professoren einer Universität abgespeichert ist. Dann ist denkbar, dass man relativ vielen Benutzern Zugriff auf die Information bezüglich Rang, Raum, Telefon, gelesene Vorlesungen, etc. gewährt. Die Information bezüglich Gehalt und abgenommener Prüfungen ist andererseits vor dem Zugriff durch die meisten Benutzer zu schützen.

Datenbankverwaltungssysteme bieten die Möglichkeit, die Zugriffsrechte sehr flexibel an einzelne Benutzer bzw. Benutzergruppen zu übertragen. Dabei ist es auch möglich, Informationsteile (z.B. Gehalt) bezüglich bestimmter Objekte (z.B. Professoren) auszublenden, während andere Teile (z.B. Telefonnummer) sichtbar bleiben.

Hohe Entwicklungskosten In vielen Fällen muss für die Entwicklung eines neuen Anwendungsprogrammes praktisch „das Rad neu erfunden“ werden. Jedesmal muss sich der Anwendungsprogrammierer zusätzlich zu Fragen der Dateiverwaltung mit zumindest einer Teilmenge der obigen Probleme auseinandersetzen. DBMS stellen eine deutlich komfortablere Schnittstelle dar, die die Entwicklungszeiten (und damit die Kosten) für neue Anwendungen verkürzt und die Fehleranfälligkeit reduziert.

1.2 Datenabstraktion

Man unterscheidet drei Abstraktionsebenen im Datenbanksystem (Abbildung 1.1):

1. *Die physische Ebene:* Auf dieser Ebene wird festgelegt, wie die Daten gespeichert sind. Im Allgemeinen sind die Daten auf dem Hintergrundspeicher (meistens als Plattenspeicher realisiert) abgelegt.
2. *Die logische Ebene:* Auf der logischen Ebene wird in einem sogenannten *Datenbankschema* festgelegt, welche Daten abgespeichert sind.

3. *Die Sichten*: Während das Datenbankschema der logischen Ebene ein integriertes Modell der gesamten Informationsmenge des jeweiligen Anwendungsbereichs (z.B. des gesamten Unternehmens) darstellt, werden in den Sichten Teilmengen der Information bereitgestellt. Die Sichten sind auf die Bedürfnisse der jeweiligen Benutzer bzw. Benutzergruppen zugeschnitten. Mögliche Benutzergruppen in den Universitäten wären etwa die Studenten, die Professoren, die Hausmeister, etc.

Die physische Ebene ist für den „normalen“ Benutzer eines Datenbanksystems nicht relevant. Auf dieser Ebene werden die Speicherstrukturen und eventuell Indexstrukturen für das schnelle Auffinden von Daten festgelegt. Die Pflege der physischen Ebene eines Datenbanksystems obliegt einem Systemprogrammierer bzw. einer Systemprogrammiererin – im Datenbankjargon als *Datenbankadministrator/in* (DBA) bezeichnet. Änderungen der physischen Speicherstruktur werden nur zum Zweck der Leistungssteigerung des DBMS vollzogen. Will man das Informationsmodell ändern – wie z.B. zusätzliche Daten einbeziehen oder zusätzliche Beziehungen zwischen Daten modellieren – muss die logische Ebene, also das Datenbankschema geändert werden. Das Datenbankschema kann man sich als eine Menge von Typdefinitionen – wie z.B. Record-Typen in Pascal – vorstellen. Diese Typdefinitionen legen die logische Struktur der Dateneinheiten fest. Auf der Ebene der Sichten können die im Datenbankschema (auf der logischen Ebene des DBMS) festgelegten Strukturen auf die besonderen Bedürfnisse bestimmter Anwender(gruppen) zugeschnitten werden. Zum einen wollen Anwender i.A. nur einen Ausschnitt des gesamten Informationsmodells zu sehen bekommen. Zum anderen können bestimmte kritische Informationsteile in den Sichten ausgeblendet werden, um dadurch den Datenschutz zu gewährleisten.

1.3 Datenunabhängigkeit

Die drei Ebenen eines DBMS gewährleisten einen bestimmten Grad der *Datenunabhängigkeit*. Dies ist analog zum Konzept der abstrakten Datentypen (ADTs) in Programmiersprachen. Durch eine wohldefinierte Schnittstelle wird die „darunterliegende“ Implementierung verdeckt, so dass man – bei Beibehaltung der Schnittstelle – die Realisierung variieren kann, ohne dass die Benutzer der Schnittstelle davon in Mitleidenschaft gezogen werden. Aufgrund der drei Schichten ergeben sich zwei Stufen der Datenunabhängigkeit im DBMS.

- *Physische Datenunabhängigkeit*: Die Modifikation der physischen Speicherstruktur belässt die logische Ebene (also das Datenbankschema) invariant. Z.B. erlauben fast alle Datenbanksysteme das nachträgliche Anlegen eines Indexes, um die Datenobjekte schneller finden zu können. Dies darf keinen Einfluss auf bereits existierende Anwendungen auf der logischen Ebene haben – außer natürlich hinsichtlich der Effizienz.
- *Logische Datenunabhängigkeit*: In den Anwendungen wird (natürlich) Bezug auf die logische Struktur der Datenbasis genommen: Es werden Mengen von Datenobjekten nach einem Namen angesprochen, die Datenobjekte haben „benannte“ Eigenschaften, etc. Man denke etwa an eine Anfrage, in der die Professoren ermittelt werden, die den Rang „C2“ haben. In einer solchen Anfrage

wird vorausgesetzt, dass es eine Menge von Professoren gibt und dass die Datenobjekte, die Professoren repräsentieren, eine Eigenschaft (Attribut, Feld) namens *Rang* haben. Bei Änderungen der logischen Ebene (also des Datenbankschemas) könnte z.B. diese Eigenschaften umbenannt werden in, sagen wir, *Gehaltsstufe*. In einer Sichtdefinition kann man solche kleineren Änderungen vor den Anwendern verbergen. Dadurch wird zu einem gewissen Grad eine logische Datenunabhängigkeit erzielt.

Die heutigen Datenbanksysteme erfüllen zumeist die physische Datenunabhängigkeit. Die logische Datenunabhängigkeit kann schon rein konzeptuell nur für einfachste Modifikationen des Datenbankschemas gewährleistet werden.

1.4 Datenmodelle

Datenbankverwaltungssysteme basieren auf einem *Datenmodell*, das sozusagen die Infrastruktur für die Modellierung der realen Welt zur Verfügung stellt. Das Datenmodell legt die Modellierungskonstrukte fest, mittels derer man ein computerisiertes Informationsabbild der realen Welt (bzw. des relevanten Ausschnitts) generieren kann. Es beinhaltet die Möglichkeit zur

- Beschreibung der Datenobjekte und zur
- Festlegung der anwendbaren Operatoren und deren Wirkung.

Das Datenmodell ist somit analog zu einer Programmiersprache: Es legt die generischen Strukturen und Operatoren fest, die man zur Modellierung einer bestimmten Anwendung ausnutzen kann. Eine Programmiersprache legt die Typkonstrukturen und Sprachkonstrukte fest, mit deren Hilfe man spezifische Anwendungsprogramme realisiert.

Das Datenmodell besteht demnach aus zwei Teilsprachen:

1. der *Datendefinitionssprache* (engl. *Data Definition Language*, DDL) und
2. der *Datenmanipulationssprache* (engl. *Data Manipulation Language*, DML).

Die DDL wird benutzt, um die Struktur der abzuspeichernden Datenobjekte zu beschreiben. Dabei werden gleichartige Datenobjekte durch ein gemeinsames Schema (analog zu einem Datentyp in Programmiersprachen) beschrieben. Die Strukturbeschreibung aller Datenobjekte des betrachteten Anwendungsbereichs nennt man das *Datenbankschema*.

Die Datenmanipulationssprache (DML) besteht aus

- der *Anfragesprache* (engl. *Query Language*) und
- der „eigentlichen“ Datenmanipulationssprache zur Änderung von abgespeicherten Datenobjekten, zum Einfügen von Daten und zum Löschen von gespeicherten Daten.

Die DML (einschließlich der Anfragesprache) kann in zwei unterschiedlichen Arten genutzt werden:

- *interaktiv*, indem DML-Kommandos direkt am Arbeitsplatzrechner (oder Terminal) eingegeben werden, oder
- in einem Programm einer höheren Programmiersprache, das „eingebettete“ DML-Kommandos enthält.

1.5 Datenbankschema und Ausprägung

Man muss sehr klar zwischen *Datenbankschema* und *Datenbankausprägung* unterscheiden: Das Datenbankschema legt die Struktur¹ der abspeicherbaren Datenobjekte fest. Das Schema sagt also noch nichts über die individuellen Datenobjekte aus. Deshalb kann man das Datenbankschema auch als *Metadaten* – also Daten über Daten – verstehen.

Unter der Datenbankausprägung versteht man demgegenüber den momentan gültigen (also abgespeicherten) Zustand der Datenbasis. Die Datenbankausprägung muss also den im Schema festgelegten Strukturbeschreibungen „gehörchen“. Manchmal spricht man in diesem Zusammenhang auch von der *intensionalen* (Schema) und der *extensionalen* (Ausprägung) Ebene einer Datenbank.

Im Allgemeinen geht man davon aus, dass sich das einmal festgelegte Datenbankschema sehr selten ändert, wohingegen die Datenbankausprägung einer laufenden Modifikation unterliegt. Man denke etwa an ein Flugbuchungssystem: Jede Reservierung entspricht einer Änderung der Datenbankausprägung. Änderungen am Schema werden oft auch als „Schemaevolution“ bezeichnet. Man beachte, dass Schemaänderungen schwerwiegende Folgen haben können: Die bereits abgespeicherten Datenobjekte können nach einer Schemaänderung eine – gemäß dem neuen Datenbankschema – inkonsistente Struktur aufweisen.

1.6 Einordnung der Datenmodelle

In Abbildung 1.2 sind die grundlegendsten Phasen der Datenmodellierung gezeigt – in Kapitel 2 werden wir detaillierter darauf eingehen.

1.6.1 Modelle des konzeptuellen Entwurfs

Man beginnt beim Datenbankentwurf mit der Abgrenzung eines Teils der „realen Welt“, um den Ausschnitt (die sogenannte *Miniwelt*) zu bestimmen, der in der Datenbank modelliert werden soll. Diese Miniwelt wird dann konzeptuell modelliert. Für die konzeptuelle Modellierung gibt es mehrere mögliche Datenmodelle:

- Entity-Relationship-Modell, auch Gegenstand-Beziehungs-Modell genannt,
- semantisches Datenmodell und
- objektorientierte Entwurfsmodelle, wie UML (siehe Kapitel 2 und 13).

¹Im objektorientierten Datenmodell legt das Schema zusätzlich auch das Verhalten (also die Operationen) der Datenobjekte fest.

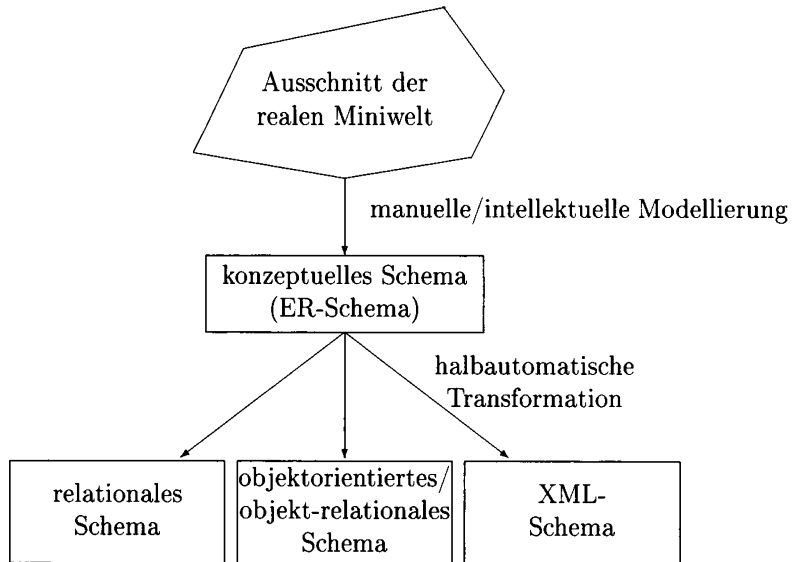


Abb. 1.2: Übersicht der Datenmodellierung

Das mit Abstand am häufigsten benutzte Modell für den konzeptuellen Entwurf ist das Entity-Relationship-Modell. Folglich werden wir uns im nächsten Kapitel detaillierter damit beschäftigen. In der konzeptuellen Entwurfsphase werden die in der realen Welt vorkommenden Konzepte in Gegenstandsmengen und Beziehungen zwischen diesen Gegenstandsmengen strukturiert.

Abbildung 1.3 zeigt diese „intellektuelle“ Aufgabe für einen ganz kleinen Ausschnitt der Universitätswelt. Es werden die Gegenstandsmengen *Studenten*, *Professoren* und *Vorlesungen* ermittelt. Weiterhin werden die Beziehungen *hören* (zwischen *Vorlesungen* und *Studenten*) und *lesen* (zwischen *Vorlesungen* und *Professoren*) bestimmt. Der untere Teil der Abbildung stellt jetzt schon ein (stark vereinfachtes) konzeptuelles Schema in der graphischen Beschreibungssprache des Entity-Relationship-Modells dar.

Die konzeptuellen Datenmodelle verfügen im Allgemeinen nur über eine DDL und haben keine Datenmanipulationssprache, da sie nur die Struktur der Daten beschreiben. Sie verzichten auf die Abbildung von individuellen Datenobjekten, d.h. es werden keine Datenbankausprägungen erzeugt. Deshalb benötigen sie natürlich auch keine Datenmodifikationssprache (DML).

1.6.2 Logische (Implementations-)Datenmodelle

Das konzeptuelle Schema ist aber in dieser Form meist nicht als Implementationschema geeignet. Die Datenmodelle für den konzeptuellen Entwurf sind i.A. reine Beschreibungsmodelle mit graphischer Notation und sehr reichhaltigen Modellierungskonstrukten, um die Gesetzmäßigkeiten der realen Welt möglichst anschaulich abbilden zu können.

Die logische Ebene eines Datenbankverwaltungssystems wird von einem der folgenden Datenmodelle gebildet:

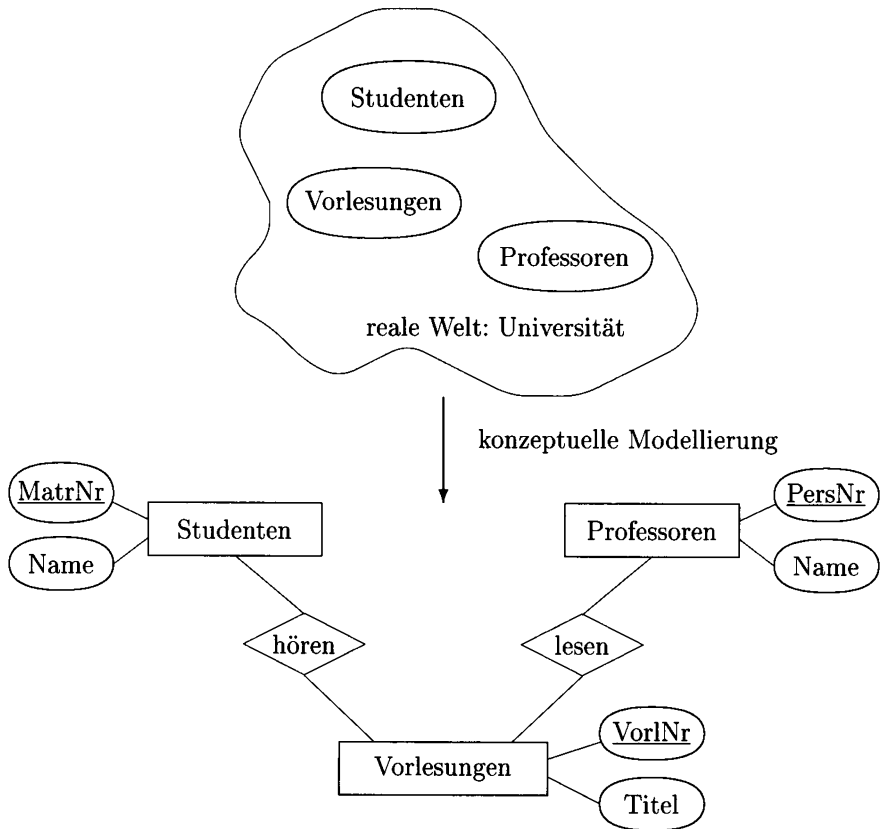


Abb. 1.3: Konzeptuelle Modellierung einer (sehr kleinen) Beispielanwendung

- Netzwerkmodell, oder hierarchisches Datenmodell,
- relationales Datenmodell (Kapitel 3),
- objektorientiertes und objekt-relationales Datenmodell (Kapitel 13 und 14),
- deduktives Datenmodell (Kapitel 15),
- XML (Kapitel 19).

Die beiden erstgenannten Datenmodelle – das Netzwerk- und das hierarchische Datenmodell – werden oft als *satzorientierte* Datenmodelle zusammengefasst. Sie haben heute fast nur noch historische Bedeutung: Es gibt aber etliche Altinstallationen von Datenbanksystemen, die noch auf diesen Datenmodellen basieren. Eine Transition zu einem „modernen“ Datenmodell ist natürlich mit hohen Kosten verbunden, da die installierten Datenbanken i.A. enorme Volumina angenommen haben. Brodie und Stonebraker (1995) behandeln dieses Problem der „legacy systems“ – frei übersetzt: Altlasten. Datenbankverwaltungssysteme, die noch auf diesen satzorientierten Datenmodellen beruhen, sind beispielsweise *IMS* von IBM basierend auf dem hierarchischen Modell und *UDS* von Siemens basierend auf dem Netzwerkmodell.

Die relationalen Datenbanksysteme sind heute marktbeherrschend und bilden folglich den Schwerpunkt in diesem Buch. Wir haben aber auch den deduktiven und den objektorientierten Datenbanken je ein Kapitel gewidmet. Das deduktive Datenmodell stellt eine Erweiterung des relationalen Modells um eine Regel- oder Deduktionskomponente dar. Formal gesehen basiert das Modell auf der Logik erster Stufe und wird deshalb manchmal auch Logik-Datenmodell genannt. Die objektorientierten Datenbanksysteme werden heute vielfach als die nächste Generation der Datenbanktechnologie angesehen. Um dieser Herausforderung zu begegnen, haben die Hersteller relationaler Systeme versucht, einige Konzepte des objektorientierten Datenmodells ins relationale Modell zu übertragen. Aus diesem Grund kann man davon ausgehen, dass die Datenbanksysteme der nächsten Generation objektorientierte Modellierungskonstrukte beinhalten.

Zunehmend werden Daten heutzutage im XML-Format modelliert, wobei XML sich besonders als Datenaustauschformat zwischen heterogenen, verteilten Anwendungen durchsetzt. Auch für XML-basierte Daten bieten kommerzielle relationale Datenbanksysteme schon weitreichende Unterstützung.

Wir wollen hier noch kurz die relationale Darstellung eines Teils unseres konzeptuellen Universitäts-Schemas (Abbildung 1.3) zeigen. Nachfolgend sind die drei Relationen *Studenten*, *hören* und *Vorlesungen* gezeigt:

Studenten		hören		Vorlesungen	
MatrNr	Name	MatrNr	VorlNr	VorlNr	Titel
26120	Fichte	25403	5022	5001	Grundzüge
25403	Jonas	26120	5001	5022	Glaube und Wissen
...	...	...	...	...	...

Relationen kann man sich als „flache“ *Tabellen* (engl. *table*) vorstellen. Die Zeilen entsprechen den Datenobjekten der realen Welt – hier also *Studenten* und *Vorlesungen*. Die Spalten geben die Eigenschaften der Datenobjekte an – z.B. die *MatrNr* (Matrikelnummer) und den *Namen* der Studenten. Die Relation *hören* nimmt eine gewisse Sonderstellung ein: Sie modelliert die Beziehung zwischen *Studenten* und *Vorlesungen*. Die Zeile [25403, 5022] gibt dabei zum Beispiel an, dass der Student namens „Jonas“ mit der *MatrNr* 25403 als Hörer an der Vorlesung mit dem Titel „Glaube und Wissen“ und der *VorlNr* (Vorlesungsnummer) 5022 teilnimmt.

Diese Tabellen stellen die logische Sicht der Datenbankbenutzer dar. Es gibt eine standardisierte DML namens SQL für die Manipulation und Abfrage dieser Tabellen. Wir wollen dem Leser hier anhand zweier Beispiele nur einen kleinen Vorgeschmack auf diese Sprache geben. Als erstes wollen wir die Namen der Studenten ermitteln, die an der Vorlesung „Grundzüge“ teilnehmen:

```

select Name
from Studenten, hören, Vorlesungen
where Studenten.MatrNr = hören.MatrNr and
        hören.VorlNr = Vorlesungen.VorlNr and
        Vorlesungen.Titel = 'Grundzüge';

```

Bei dieser Anfrage werden die Inhalte der Relationen *Studenten*, *hören* und *Vorlesungen* kombiniert (verknüpft), um die gewünschte Information aus der Datenbank zu extrahieren. Für eine genauere Erläuterung verweisen wir auf Kapitel 4.

Im nächsten Beispiel wollen wir den Titel der Vorlesung mit der *VorlNr* 5001 in „Grundzüge der Logik“ ändern:

```
update Vorlesungen  
  set Titel = 'Grundzüge der Logik'  
  where VorlNr = 5001;
```

In der **where**-Klausel wird also die zu ändernde Zeile bestimmt und in der **set**-Klausel wird der neue Wert der *Titel*-Spalte angegeben. Man kann in SQL auch mehrere Zeilen und/oder Spalten gleichzeitig ändern.

Wir sollten hier nochmals den Zusammenhang zwischen Datenbank-Schema und -Ausprägung betonen. Die Zeilen der Relationen stellen die Datenbankausprägung dar, die dem momentanen Zustand der Datenbasis entspricht. Die Struktur der Tabellen – also die Anzahl der Spalten, die Benennung der Spalten, die zulässigen Wertemengen für die Spalten, etc. – stellt das Datenbankschema dar, das nur sehr selten (wenn überhaupt) geändert wird.

1.7 Architekturübersicht eines DBMS

Abbildung 1.4 zeigt eine stark vereinfachte Darstellung der Architektur eines Datenbankverwaltungssystems. Im oberen Bereich befindet sich die Benutzerschnittstelle. Je nach Erfahrung und Verantwortlichkeit greifen unterschiedliche Benutzergruppen auf unterschiedliche Schnittstellen zu:

- Für häufig erledigte und immer ähnliche Aufgaben werden speziell abgestimmte Anwendungsprogramme zur Verfügung gestellt. Diese Anwendungsprogramme sind leichter zu erlernen und effizienter zu bedienen als eine komplette Anfragesprache. Oft sind diese Anwendungssysteme über eine Menü-gesteuerte Benutzerschnittstelle ausführbar – man denke etwa an ein Flugreservierungssystem, das von den Angestellten eines Reisebüros bedient wird.
- Fortgeschrittene Benutzer mit ständig wechselnden Aufgaben können interaktiv Anfragen in einer flexiblen Anfragesprache (wie SQL) eingeben.
- Anwendungsprogrammierer können durch „Einbettung“ von Elementen der Anfragesprache in eine Programmiersprache besonders komplexe Datenverarbeitungsanforderungen erfüllen oder weniger geschulten Benutzern einfach zu bedienende Anwendungsprogramme zur Verfügung stellen. Dieser Mechanismus wird in Kapitel 4 besprochen.
- Die Schnittstelle für die Datenbankadministration ermöglicht unter anderem die Manipulation des Schemas und das Anlegen von Benutzerkennungen.

Anforderungen von Daten durch die Benutzer werden zunächst vom DML-Compiler untersucht und in eine für die Anfragebearbeitung verständliche Form gebracht. Die Anfragebearbeitung (siehe Kapitel 8) untersucht, wie die Anforderung effizient erfüllt werden kann und wandelt sie in Unterprogrammaufrufe des Datenbankmanagers um. Der Datenbankmanager ist das Kernstück des DBMS: Hier werden die Anfragen ausgeführt. Er bildet die Schnittstelle zur Dateiverwaltung.

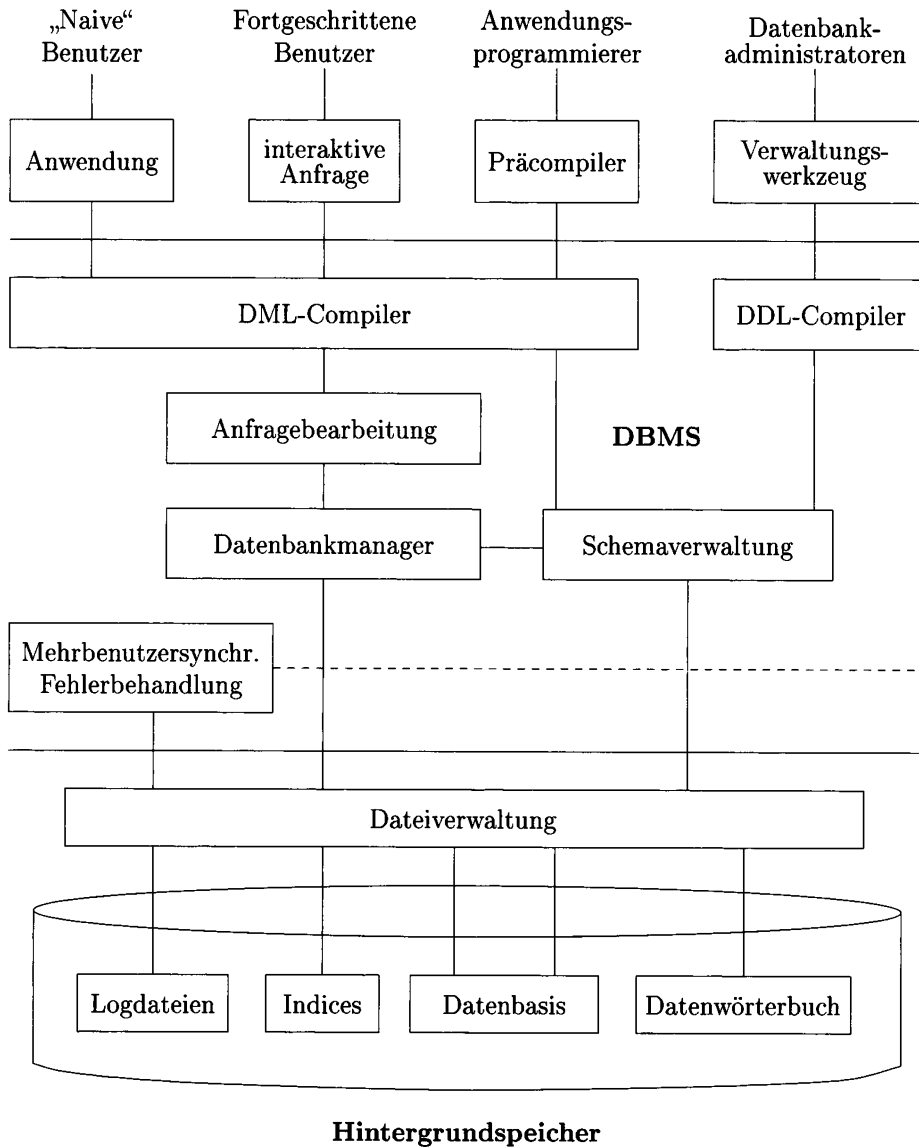


Abb. 1.4: Architekturübersicht eines DBMS

Schemamanipulationen durch den DBA werden vom DDL-Compiler analysiert und in Metadaten übersetzt. Diese Metadaten werden von der Schemaverwaltung verarbeitet und im sogenannten Datenwörterbuch gespeichert.

Das für die Mehrbenutzersynchronisation und für die Fehlerbehandlung zuständige Modul (siehe Kapitel 9 bis Kapitel 11) verhindert die Zerstörung von Daten und ist für das Anlegen von Archivkopien und Protokollinformationen zuständig.

1.8 Übungen

- 1.1 In Abschnitt 1.1 haben wir davon gesprochen, dass unkontrollierte Redundanz unerwünscht ist. Können Sie sich eine sinnvolle Einsatzmöglichkeit für eine durch das DBMS kontrollierte Redundanz vorstellen?
- 1.2 In einer Universität soll ein DBMS eingesetzt werden. Überlegen Sie sich, welche Daten in einer Universität anfallen, welche Benutzergruppen es gibt und welche Anwendungsprogramme sinnvoll wären. Wie würde die notwendige Funktionalität ohne DBMS realisiert werden? Untersuchen Sie an konkreten Beispielen die in diesem Kapitel beschriebenen Probleme.
- 1.3 Konzipieren Sie ein Wahlinformationssystem für Bundestagswahlen.

1.9 Literatur

Die meisten einführenden Bücher über Datenbanksysteme sind englischsprachig – eine Ausnahme bildet das sehr umfangreiche Buch von Vossen (2008). Ein weiteres deutschsprachiges Buch ist von Schlageter und Stucky (1983), das aber mittlerweile etwas „in die Jahre gekommen ist“. Neumann (1996) und Kleinschmidt und Rank (2002) haben sehr praxisnahe Bücher über die relationale Datenbankanwendung verfasst. Lockemann, Krüger und Krumm (1993) versuchen, in einer Einführungsveranstaltung die grundlegenden Konzepte aus dem Datenbank- und dem Telekommunikationsbereich zu vereinen. Das Buch von Biskup (1995) hat einen etwas stärkeren theoretischen „Touch“. Saake, Sattler und Heuer (2007) betonen den Datenmodellierungsaspekt stärker. Lausen (2005) vermittelt die Grundlagen relationaler Datenbanken sowie der XML-Datenbanktechnologie. Das Buch von Silberschatz, Korth und Sudarshan (2005) kommt vom Lehrinhalt diesem Buch am nächsten. Die beiden Bücher von Ullman [Ullman (1988); Ullman (1989)] haben einen stärkeren theoretischen „Einschlag“ – insbesondere wird das deduktive Datenmodell sehr ausführlich behandelt. Das sehr pragmatische Buch von Date (2003) ist schon fast ein Klassiker; es ist mittlerweile schon in der sechsten Auflage erschienen. Das Buch von Elmasri und Navathe (2006) ist sehr umfangreich und detailliert. Allen Lesern, die sich intensiver mit der Datenbankforschung auseinandersetzen möchten, sei der hervorragende Bibliographieserver von M. Ley (<http://dblp.uni-trier.de/>) in Trier empfohlen. Weiterhin sei die Mitgliedschaft bei ACM SIGMOD (Association for Computing Machinery, Special Interest Group on Management of Data, <http://www.acm.org>) empfohlen.